

AI Usage Log — Sprint 1

Project Title: LaunchpadAI — Job Search and Application Tracking Platform[cite: 1]

Course: SOEN 341 Software Process (Fall 2026)[cite: 1]

Team Member: Sudhansu Mohanty[cite: 1]

Repository Directory: AI_Log/sudhansu_mohanty/[cite: 1]

Log Entry 1: Elicitation & Definition of US-01 (Account Registration & Roles)

- **Task ID/Title:** Task 1.0 / US-01: User Registration with Role Selection[cite: 1, 6]
- **Purpose of AI Use:** Brainstorming, initial requirements elicitation, and task breakdown for user registration[cite: 1].
- **Chat Link or Prompt/Response:**
 - **Tool:** Gemini / ChatGPT[cite: 1]
 - **Link:** [Insert Chat Link if available, otherwise refer to Appendix][cite: 1]
 - **Prompt:**

"Act as an Agile product manager and software requirements analyst. We are building LaunchPadAI for SOEN 341, a web-based platform designed to help job seekers manage their search activities and recruiters manage candidate applications.

Primary Users:

1. Job Seekers: Need to create profiles, upload resumes, search jobs, submit applications, and track progress (Applied, Interview, Offered,

Rejected).

2. Recruiters: Need to post jobs, manage job listings, and review incoming candidate pipelines.

Generate 10 distinct, structured user stories following the Agile template ('As a [role], I want to [action], so that [outcome]'). For each story, provide 2 to 4 concrete acceptance criteria and 2 to 3 subtasks covering database, backend API, and frontend implementation."[cite: 1]

○ AI Response (Raw US-01 Output):

User Story: As a platform visitor, I want to register for an account using my email and password and select my role (Job Seeker or Recruiter), so that I can access the correct platform features for my needs[cite: 1, 6].

Acceptance Criteria:

- Users can sign up with an email and password.
- Users choose between 'Job Seeker' and 'Recruiter' via a radio button.
- Successful signup sends a confirmation email and logs the user in.

Tasks:

- Create user database table.
- Implement signup API endpoint.
- Build registration form interface.

- **AI-Suggested Content:** Baseline registration story, fundamental role toggle concept, and standard authentication subtasks[cite: 1, 6].
- **Validation:**
 - Cross-referenced against the SOEN 341 Sprint 1 recommended demonstration deliverables (Feature 1: User Registration/Login)[cite: 1].
 - Evaluated technical feasibility against Supabase Auth, identifying that custom user data and roles belong in a

- linked `public.profiles` table rather than the protected `auth.users` schema[cite: 5].
 - Reviewed during team meeting; confirmed that acceptance criteria needed explicit validation rules (e.g., password length, unique email error handling)[cite: 6].
 - **Decision: Modified before use**[cite: 1].
 - Rewrote acceptance criteria to include PostgreSQL automated trigger execution (`handle_new_user`) in Supabase to sync identity metadata into `public.profiles`[cite: 5, 6].
 - Restructured subtasks into concrete GitHub Issues: **Task 1.1** (SQL migration & role enum trigger), **Task 1.2** (Frontend registration form UI), and **Task 1.3** (Wiring Supabase Auth client with role redirects)[cite: 5, 6].
 - **Reflection:** The generated output provided a functional starting point, but it was generic[cite: 1]. Adapting it required specifying our Supabase architecture, Row Level Security (RLS) constraints, and edge-case handling to make the tasks actionable for Sprint 1 development[cite: 1, 5].
 - **Responsible Person:** Sudhansu Mohanty[cite: 1]
-

Log Entry 2: Elicitation & Definition of US-04 (Resume Upload & Storage)

- **Task ID/Title:** Task 1.0 / US-04: Resume Document Upload & Storage[cite: 1, 6]
- **Purpose of AI Use:** Requirements elicitation, storage architecture scoping, and task decomposition for resume handling[cite: 1].
- **Chat Link or Prompt/Response:**
 - **Tool:** Gemini / ChatGPT[cite: 1]
 - **Link:** [Insert Chat Link if available, otherwise refer to Appendix][cite: 1]
 - **Prompt:** (*Elicited as part of the shared 10-story prompt referenced in Log Entry 1*)[cite: 1]
 - **AI Response (Raw US-04 Output):**

User Story: As a job seeker, I want to upload my resume to my account, so that recruiters can review my qualifications when I apply for jobs[cite: 1, 6].

Acceptance Criteria:

- Candidate can select and upload a resume document from their device.
- The file name is visible on their profile.
- The uploaded file can be deleted or replaced.

Tasks:

- Configure file storage on server.
- Build file upload dropzone UI.
- Link file URL to candidate profile.

- **AI-Suggested Content:** General user story template for file uploading, visibility on profile, and basic replacement capabilities[cite: 1, 6].
- **Validation:**
 - Verified against Sprint 1 demo requirements (Feature 2: Resume Upload / Profile Management)[cite: 1].
 - Security check: The raw output lacked access control constraints and format verification, creating vulnerabilities around unrestricted uploads and cross-user data access[cite: 5, 6].
 - Stack alignment: Evaluated Supabase Storage bucket capabilities and determined that an RLS policy scoped to `auth.uid()` was necessary to isolate resumes[cite: 5, 6].
- **Decision: Modified before use**[cite: 1].
 - Added explicit acceptance criteria: limit file types to PDF only, enforce a 5 MB file size limit, and require storage inside a private Supabase Storage bucket[cite: 5, 6].
 - Formatted subtasks into repository work items: **Task 4.1** (Provision private bucket with authenticated RLS policy), **Task 4.2** (Client-side drag-and-drop UI with format/size validation), and **Task 4.3** (Upload API integration and metadata persistence to profile)[cite: 5, 6].
- **Reflection:** AI suggested the overall user need, but failed to

address security, storage limits, and access controls[cite: 1].

Refining the story ourselves ensured that data isolation and RLS policies were accounted for prior to implementation[cite: 1, 5].

- **Responsible Person:** Sudhansu Mohanty[cite: 1]
-

Log Entry 3: Sprint 1 Scope Analysis and Feature Planning

- **Date:** 2026-09-26
- **Task ID/Title:** Sprint 1 Feature Scoping
- **Purpose of AI Use:** Analyzing the Sprint 1 project brief to identify required code deliverables and scope frontend vs. backend work.
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompt:**

"Analyse sprint 1 details and let me know what are the code implementations of all its in class-resources."

Follow-up prompts clarified scope: resume upload for Sprint 1, program management deferred; Supabase chosen for auth and database; app renamed to LaunchPadAI.

- **AI Response Summary:** Claude identified the two required Sprint 1 demo features (User Registration/Login and Resume Upload) from the project brief PDF. It recommended scoping frontend-only work first, then layering in Supabase Auth and Storage.
- **Validation:** Cross-referenced against `class-resources/` `SOEN341_CareerConnect_Project_Sprint1.pdf` to confirm minimum deliverables.
- **Decision:** Accepted AI's feature scoping. Deferred GenAI resume scoring to a future sprint. Proceeded with frontend-first approach.
- **Reflection:** AI helped quickly parse the project brief and prioritize which features were mandatory for the Sprint 1 demo versus stretch goals.

- **Responsible Person:** Sudhansu Mohanty
-

Log Entry 4: Frontend UI Development and GitHub Pages Debugging

- **Date:** 2026-09-26 to 2026-09-27
- **Task ID/Title:** Frontend UI Build and Deployment Fixes
- **Purpose of AI Use:** Building the frontend UI components (login/register pages, homepage layout, testimonial panel) and debugging GitHub Pages deployment errors (HTTP 500, routing issues).
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompts (summarized across session):**
 - "A few changes to the login and sign up page, make it a horizontal rectangle and on the right 50% there are some random testimonials that rotate and fade away."
 - "Comment the see your resume through a recruiter lens, and the FAQ section."
 - "In the how it works section, the upload resume should be 3rd step instead of 2nd and make the numbers more visible and just white."
 - "There's a weird padding in the steps 1 and 2, make sure the boxes are identical in structure."
 - "Once logged in, remove the get started and sign in button on the home page."
 - "Make sure the animation is smooth when it pans down."
 - Debugging: "I'm getting code 500", "The / dashboard/resume doesn't open on the same folder so it doesn't exist", "Why was there an issue with the GitHub Pages?"
 - **AI Response Summary:** Claude generated the

`TestimonialPanel` component, restructured the homepage steps, fixed CSS padding inconsistencies, added conditional rendering for authenticated users, and diagnosed GitHub Pages routing issues (missing `basePath` configuration and environment variable injection in the CI workflow).

- **Validation:** Each UI change was visually verified in the local dev server. GitHub Pages deployment was tested after each fix by pushing and checking the live URL.
 - **Decision:** Accepted all UI code generated by AI. Modified the homepage step ordering manually based on team preference. AI's GitHub Pages fix (adding `basePath` and env secrets to the workflow) was adopted directly.
 - **Reflection:** AI was highly effective for rapid UI iteration and debugging build/deployment issues. The 500 error and routing problems would have taken significantly longer to diagnose manually. However, trivial follow-up prompts could have been consolidated into fewer, more specific requests.
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 5: Supabase Authentication Integration

- **Date:** 2026-09-27
- **Task ID/Title:** Supabase Auth Client Setup and Session Management
- **Purpose of AI Use:** Configuring Supabase client credentials, implementing sign-up/login flows, session persistence, and post-login UX (dashboard page, conditional navbar).
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompts (summarized across session):**

■ "Now we can start integrating it to the Supabase. Right now for Sprint 1, just auth and maybe database for storing the emails and login info."

- "Teach me how it [Supabase auth] works."
- Provided Supabase project URL and anon key for client configuration (credentials stored in environment variables, not hardcoded).
- "Yes turn off email confirmation, let's test it but before that let's make a page for when the user logs in."
- "I think once we log in, it should save the session? How to do that?"
- "How does it handle the frontend when we sign in? I can still see get started or sign-in."
- "I just realized I should have an option to choose either you're a job-seeker or recruiter."
- "On the register page just a toggle option like, 'I'm a' recruiter or job-seeker."
- "How is it saved in the backend?"

○ **AI Response Summary:** Claude configured the `supabase.ts` client, implemented `signUp()` and `signInWithPassword()` calls, added session persistence using `onAuthStateChange`, created the dashboard page with logout functionality, and added a role selector toggle to the registration form. It also explained the `public.profiles` table with the `handle_new_user()` PostgreSQL trigger for storing roles.

- **Validation:** Tested registration and login locally with Supabase dashboard open to verify user records and profile rows were created correctly. Confirmed role values were persisted in `public.profiles`.
 - **Decision:** Accepted AI-generated auth code. Modified the role selector styling to match the existing design language. The `handle_new_user()` trigger SQL was reviewed and applied manually in the Supabase SQL editor.
 - **Reflection:** AI provided a complete end-to-end auth implementation quickly. The explanation of Supabase's `auth.users` vs `public.profiles` architecture was helpful for understanding the data model. Credentials shared in prompts should have been handled more carefully (e.g., via environment variables from the start rather than pasting inline).
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 6: Resume Upload Wiring to Supabase Storage

- **Date:** 2026-09-27
 - **Task ID/Title:** Resume Storage Integration
 - **Purpose of AI Use:** Connecting the existing drag-and-drop resume upload UI to Supabase Storage with user-scoped access policies.
 - **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompts:**
 - "How is the resume upload feature working out right now? Is it being stored somewhere profile-wise?"
 - "Wire up the resume upload to Supabase storage."
 - **AI Response Summary:** Claude identified that the existing `ResumeUploader` component was client-side only (no persistence). It then implemented `supabase.storage.from('resumes').upload()` with the file path scoped to `{user.id}/resume.pdf`, added PDF-only and 5 MB validation, and updated the profile record with the uploaded file path.
 - **Validation:** Tested upload locally; verified file appeared in Supabase Storage dashboard under the correct user folder. Confirmed RLS policy restricted access to the authenticated user's own files.
 - **Decision:** Accepted AI-generated storage integration code. Storage bucket and RLS policies were provisioned manually in Supabase dashboard following AI's instructions.
 - **Reflection:** AI correctly identified the gap between the UI component and actual persistence, and provided a working integration quickly. The RLS policy recommendation (`resumes/{auth.uid()}/*`) was critical for data isolation.
 - **Responsible Person:** Sudhansu Mohanty
-

Log Entry 7: Backlog Traceability and GitHub Issue Automation

- **Date:** 2026-09-27
- **Task ID/Title:** Sprint 1 GitHub Issue Creation and Commit Linking
- **Purpose of AI Use:** Automating the creation of 4 technical task issues on GitHub with commit SHA traceability, milestone assignment, and proper labeling for TA review.
- **Prompt/Response:**
 - **Tool:** Claude Code (CLI)
 - **Prompt:** Provided a detailed instruction set to inspect `git log`, verify/create labels and the Sprint 1 milestone, create 4 issues (Frontend Scaffold, Supabase Backend, Auth & Routing, Resume Storage) with commit SHAs in each body, and close them with resolution comments.
 - **AI Response Summary:** Claude ran `git log --oneline -n 15`, mapped commits to each issue (`fbdc9b1`, `1ca35e2`, `48b50fb`, `a9f9e8c`), created the Sprint 1 milestone and 8 labels, generated 4 issues (#19-#22) via `gh issue create`, and closed each with a verification comment.
- **Validation:** Verified issues appeared on GitHub under the Sprint 1 milestone with correct labels, assignees, and commit references.
- **Decision:** Accepted all generated issues. The commit-to-issue mapping was reviewed and confirmed accurate.
- **Reflection:** AI automated what would have been a tedious manual process of creating structured issues with traceability. The prompt was highly specific, which resulted in accurate output with no modifications needed.
- **Responsible Person:** Sudhansu Mohanty

Date: 2026-09-27

Prompt:

how to push to a different branch